

DSA Array Mastery Handbook

Table of Contents

Chapter 1 - Array Basics	p.2
Chapter 2 - Two Pointers	p.9
Chapter 3 - Sliding Window	p.18
Chapter 4 - Prefix Sum & Hashing	p.29
Chapter 5 - Kadane's Algorithm & Array Rearrangement	p.42
Chapter 6 - Matrix	p.55
Chapter 7 - Binary Search on Arrays	p.69
Chapter 8 - Sorting	p.81

DSA Array Mastery Handbook

Chapter 1 — Array Basics

Table of Contents

- Problem 1: Largest Element in an Array
- Problem 2: Second Largest Element
- Problem 3: Check if Array is Sorted
- Problem 4: Remove Duplicates from Sorted Array
- Problem 5: Left Rotate Array by One
- Problem 6: Move Zeroes

Chapter 1 — Array Basics

Problems

- Largest Element in an Array
- Second Largest Element
- Check if Array is Sorted
- Remove Duplicates from Sorted Array
- Left Rotate Array by One
- Move Zeroes

Problem 1: Largest Element in an Array

Difficulty: Easy

Pattern: Array Traversal

Problem

Find the largest element in an array.

Input: [3, 5, 1, 8, 2]
Output: 8

★ Intuition

The simplest thought is to sort the array and take the last element. But we don't actually need to know the order of all elements.

We only care about one thing:

"What is the largest value I have seen so far?"

Start with the first element as the current maximum. Traverse the array and whenever you find a bigger element, update the maximum.

So we can solve the problem in a single traversal without modifying the array.

Brute

Sort the array and take the last element.

```
int largest(vector<int>& a)
{
    sort(a.begin(), a.end());
```

```
    return a.back();  
}
```

Time: $O(N \log N)$

Space: $O(1)$

Optimal ☆

```
int largest(vector<int>& a)  
{  
    int maxi = a[0];  
  
    for(int x : a)  
        maxi = max(maxi, x);  
  
    return maxi;  
}
```

Time: $O(N)$

Space: $O(1)$

Interview Tip

If you only need the maximum/minimum, don't sort unless you need ordering.

Problem 2: Second Largest Element

Difficulty: Easy

Pattern: Traversal

Problem

Find the second largest distinct element in an array.

Input: [10, 5, 8, 10, 3]
Output: 8

☆ Intuition

Sorting makes this easy, but again we don't need the complete ordering.

We only need to maintain:

largest
second largest

Whenever a new element becomes the largest, the old largest becomes the second largest.

If an element is smaller than largest but larger than second, it becomes the new second largest.

The important detail is distinct: if the array contains [10,10,8], the answer is 8, not 10.

Brute

Sort and scan from the end to find a different element.

```
int secondLargest(vector<int>& a)  
{  
    sort(a.begin(), a.end());  
  
    for(int i=a.size()-2; i>=0; i--)  
        if(a[i] != a.back())  
            return a[i];  
  
    return -1;  
}
```

Time: $O(N \log N)$

Better

Find the largest first, then make another traversal to find the largest element smaller than it.

Time: $O(N)$

Space: $O(1)$

Optimal ★

Find both values in one traversal.

```
int secondLargest(vector<int>& a)
{
    int largest = INT_MIN;
    int second = INT_MIN;

    for(int x : a)
    {
        if(x > largest)
        {
            second = largest;
            largest = x;
        }
        else if(x > second && x != largest)
        {
            second = x;
        }
    }

    return second;
}
```

Time: $O(N)$

Space: $O(1)$

Common Mistake

Don't write:

```
else if(x > second)
```

without checking `x != largest`, because duplicates would incorrectly become the second largest.

Problem 3: Check if Array is Sorted

Difficulty: Easy

Pattern: Traversal

Problem

Check whether an array is sorted in non-decreasing order.

```
[1,2,2,4,5] → true
```

```
[1,3,2,4] → false
```

★ Intuition

A sorted array has one simple property:

Every element must be greater than or equal to the element immediately before it.

Therefore, we don't need to compare every pair.

Just compare adjacent elements:

```
a[0] <= a[1]
a[1] <= a[2]
a[2] <= a[3]
...
```

The moment we find:

```
a[i] > a[i+1]
```

the array is not sorted.

Brute

Compare every pair $i < j$.

This is unnecessary and takes $O(N^2)$.

Optimal ☆

```
bool isSorted(vector<int>& a)
{
    for(int i=1; i<a.size(); i++)
    {
        if(a[i] < a[i-1])
            return false;
    }

    return true;
}
```

Time: $O(N)$

Space: $O(1)$

Interview Tip

For checking sortedness, adjacent comparison is sufficient.

Problem 4: Remove Duplicates from Sorted Array

Difficulty: Easy

LeetCode: 26

Pattern: Two Pointers

Problem

Remove duplicates in-place from a sorted array and return the number of unique elements.

Input:
[1,1,2,2,3]

Output:
3

Array:
[1,2,3,...]

☆ Intuition

Because the array is already sorted, duplicates always appear next to each other.

So we don't need a HashSet.

Maintain:

```
i → position of the last unique element
j → scans the array
```

Whenever $a[j]$ is different from the previous unique element, place it at the next unique position.

The key observation is:

Sorted array $\rightarrow$ duplicates are adjacent $\rightarrow$ compare with the previous unique element.

Brute

Use a set to store unique values.

```
int removeDuplicates(vector<int>& a)
{
    set<int> st(a.begin(), a.end());

    int i=0;
    for(int x : st)
        a[i++] = x;

    return i;
}
```

Time: $O(N \log N)$

Space: $O(N)$

Optimal ☆

```
int removeDuplicates(vector<int>& a)
{
    if(a.empty()) return 0;

    int j=0;

    for(int i=1; i<a.size(); i++)
    {
        if(a[i] != a[j])
            a[++j] = a[i];
    }

    return j+1;
}
```

Time: $O(N)$

Space: $O(1)$

Interview Tip

Whenever you see:

Sorted array + remove duplicates + in-place

Think Two Pointers immediately.

Problem 5: Left Rotate Array by One

Difficulty: Easy

Pattern: Array Manipulation

Problem

Rotate the array one position to the left.

Input:
[1,2,3,4,5]

Output:
[2,3,4,5,1]

★ Intuition

A left rotation by one means:

```
The first element moves to the end.  
Every other element moves one position left.
```

We can save the first element, shift everything left, and finally put the saved element at the end.

Optimal

```
void leftRotate(vector<int>& a)  
{  
    int first = a[0];  
  
    for(int i=1; i<a.size(); i++)  
        a[i-1] = a[i];  
  
    a.back() = first;  
}
```

Time: $O(N)$

Space: $O(1)$

Interview Tip

This is the foundation for understanding rotation by K, where the reversal technique becomes important.

Problem 6: Move Zeroes

Difficulty: Easy

LeetCode: 283

Pattern: Two Pointers

Problem

Move all zeroes to the end while maintaining the relative order of non-zero elements.

```
Input:  
[0,1,0,3,12]  
  
Output:  
[1,3,12,0,0]
```

★ Intuition

We don't actually need to "move every zero".

Instead, think:

Collect all non-zero elements at the front.

Maintain a pointer j representing the position where the next non-zero element should go.

Scan the array:

```
If the current element is non-zero, swap it with a[j].  
Move j forward.
```

At the end, all non-zero elements are at the front and all zeroes naturally end up at the back.

This is the same in-place two-pointer idea we saw in Remove Element.

Brute

Create a temporary array containing all non-zero elements and then append zeroes.

```
void moveZeroes(vector<int>& a)
{
    vector<int> temp;

    for(int x : a)
        if(x != 0)
            temp.push_back(x);

    while(temp.size() < a.size())
        temp.push_back(0);

    a = temp;
}
```

Time: $O(N)$

Space: $O(N)$

Optimal ☆

```
void moveZeroes(vector<int>& a)
{
    int j=0;

    for(int i=0; i<a.size(); i++)
    {
        if(a[i] != 0)
            swap(a[i], a[j++]);
    }
}
```

Time: $O(N)$

Space: $O(1)$

DSA Array Mastery Handbook

Chapter 2 — Two Pointers

Table of Contents

- 🎯 Goal
- 🔑 Main Pattern
- ★ Most Important 6 Problems
- 1. Two Sum II — Sorted Array
- 2. Remove Element
- 3. Container With Most Water
- 4. Sort Colors — Dutch National Flag
- 5. 3Sum
- 6. Trapping Rain Water
- 🧠 Chapter 2 — What You Should Remember

Chapter 2 — Two Pointers

🎯 Goal

Learn how to use two or more indices to eliminate unnecessary comparisons and reduce time complexity.

🔑 Main Pattern

The most important idea is:

Instead of checking every possible pair, use the relationship between elements to decide which pointer can be safely moved.

There are 3 major forms you'll see:

1. Opposite-direction pointers

L → ← R

Usually used when the array is sorted or when comparing both ends.

Examples: Two Sum II, Container With Most Water.

2. Same-direction pointers

i → → →
j → → →

Usually used for in-place modification/filtering.

Examples: Remove Element, Remove Duplicates.

3. Multiple pointers after sorting

Fix one/two elements and use two pointers for the remaining part.

i | j | L → ← R

Examples: 3Sum, 4Sum.

★ Most Important 6 Problems

For your handbook, I recommend these 6 core problems:

#	Problem	Pattern	Importance
1	Two Sum II	Opposite pointers	★★★★★
2	Remove Element	Same-direction	★★★★
3	Container With Most Water	Opposite + Greedy	★★★★★
4	Sort Colors	Three pointers	★★★★★
5	3Sum	Sort + Two Pointers	★★★★★
6	Trapping Rain Water	Two Pointers	★★★★★

These six cover the most important variations without unnecessarily making the chapter huge.

1. Two Sum II — Sorted Array

LeetCode 167

Problem

Find two numbers in a sorted array whose sum equals target.

```
[2,7,11,15], target = 9
→ [2,7]
```

★ Intuition

Because the array is sorted, start with the smallest and largest elements.

If their sum is:

```
Too small → we need a bigger number → left++
Too large → we need a smaller number → right--
Equal → answer found.
```

The important insight is that sorting allows us to eliminate many possibilities at once.

Brute — O(N²)

```
vector<int> twoSum(vector<int>& a, int target)
{
    for(int i=0;i<a.size();i++)
        for(int j=i+1;j<a.size();j++)
            if(a[i]+a[j]==target)
                return {i+1,j+1};

    return {};
}
```

Optimal — O(N)

```
vector<int> twoSum(vector<int>& a, int target)
{
    int l=0,r=a.size()-1;

    while(l<r)
    {
```

```

        int sum=a[l]+a[r];

        if(sum==target) return {l+1,r+1};
        if(sum<target) l++;
        else r--;
    }

    return {};
}

```

Space: $O(1)$

Key takeaway

Sorted + pair → Two Pointers

2. Remove Element

LeetCode 27

Problem

Remove all occurrences of val in-place.

```

[3,2,2,3], val=3
→ [2,2]

```

★ Intuition

We don't actually need to delete elements.

Instead, overwrite unwanted elements.

Maintain j as the position where the next valid element should go.

```

i → scans
j → places valid elements

```

Whenever $a[i] \neq \text{val}$, put it at $a[j]$.

Brute — $O(N)$ / $O(N)$

Use another array.

Optimal — $O(N)$ / $O(1)$

```

int removeElement(vector<int>& a,int val)
{
    int j=0;

    for(int i=0;i<a.size();i++)
        if(a[i]!=val)
            a[j++]=a[i];

    return j;
}

```

Key takeaway

In-place filtering → slow pointer + fast pointer

3. Container With Most Water

Problem

Find the maximum water that can be contained between two vertical lines.

```
[1,8,6,2,5,4,8,3,7]
```

```
Answer = 49
```

★ Intuition

Area is:

```
Area = min(height[L], height[R]) × (R - L)
```

Start with the widest possible container.

The shorter line limits the amount of water.

Therefore:

```
If height[L] < height[R], move L.  
Otherwise move R.
```

Why?

Moving the taller line cannot help because the shorter line is still limiting the height, while width becomes smaller.

Brute — $O(N^2)$

```
int maxArea(vector<int>& h)
{
    int ans=0;

    for(int i=0;i<h.size();i++)
        for(int j=i+1;j<h.size();j++)
            ans=max(ans,min(h[i],h[j])*(j-i));

    return ans;
}
```

Optimal — $O(N)$

```
int maxArea(vector<int>& h)
{
    int l=0,r=h.size()-1,ans=0;

    while(l<r)
    {
        ans=max(ans,min(h[l],h[r])*(r-l));

        if(h[l]<h[r]) l++;
        else r--;
    }

    return ans;
}
```

Space: $O(1)$

Key takeaway

When one side is the limiting factor, move the limiting pointer.

4. Sort Colors — Dutch National Flag

Problem

Sort an array containing only 0, 1, and 2 without using sorting.

```
[2,0,2,1,1,0]
→ [0,0,1,1,2,2]
```

★ Intuition

Because there are only three values, we can divide the array into regions:

```
0s | 1s | unknown | 2s
```

Use three pointers:

```
low  → position for 0
mid  → current element
high → position for 2
```

For `nums[mid]`:

0: swap with low, move both.

1: simply move mid.

2: swap with high, move only high.

Why don't we move mid after a 2?

Because the element coming from high has not been checked yet.

Brute — $O(N \log N)$

```
sort(nums.begin(),nums.end());
```

Better — Counting — $O(N)$

```
int cnt[3]={};

for(int x:nums) cnt[x]++;

int k=0;

for(int x=0;x<3;x++)
    while(cnt[x]--)
        nums[k++]=x;
```

Optimal — $O(N)$, $O(1)$

```
void sortColors(vector<int>& a)
{
    int low=0,mid=0,high=a.size()-1;

    while(mid<=high)
    {
        if(a[mid]==0)
            swap(a[low++],a[mid++]);

        else if(a[mid]==1)
            mid++;

        else
            swap(a[mid],a[high--]);
    }
}
```

Key takeaway

Three categories → Three-way partitioning.

5. 3Sum

LeetCode 15

Problem

Find all unique triplets whose sum is 0.

```
[-1,0,1,2,-1,-4]
```

```
→ [-1,-1,2]
```

```
→ [-1,0,1]
```

★ Intuition

Brute force checks three elements → $O(N^3)$.

Instead, fix one element.

Now we need:

```
a + b + c = 0
```

```
b + c = -a
```

So the remaining problem becomes Two Sum.

Sort the array first so that two pointers can be used.

Fix i

```
L →          ← R
Sum too small → L++
Sum too large → R--
Sum equal → store triplet
```

We also skip duplicates because the answer must contain unique triplets.

Brute — $O(N^3)$

Three nested loops.

Better — Hashing — $O(N^2)$

Fix one element and use a HashSet for the remaining two-sum search.

Optimal — $O(N^2)$

```
vector<vector<int>> threeSum(vector<int>& a)
{
    vector<vector<int>> ans;
    sort(a.begin(),a.end());

    for(int i=0;i<a.size();i++)
    {
        if(i>0 && a[i]==a[i-1]) continue;

        int l=i+1,r=a.size()-1;

        while(l<r)
        {
            int sum=a[i]+a[l]+a[r];
```

```

        if (sum < 0) l++;
        else if (sum > 0) r--;
        else
        {
            ans.push_back({a[l], a[l], a[r]});

            l++;
            r--;

            while (l < r && a[l] == a[l-1]) l++;
            while (l < r && a[r] == a[r+1]) r--;
        }
    }
}

return ans;
}

```

Key takeaway

3Sum
 ↓
 Fix 1 element
 ↓
 Two Sum with Two Pointers

This is the foundation for 4Sum and k-Sum.

6. Trapping Rain Water

LeetCode 42

Problem

Calculate how much water can be trapped between buildings.

```
[0,1,0,2,1,0,1,3,2,1,2,1]
```

Answer = 6

★ Intuition

Water at an index depends on:

```
min(leftMax, rightMax) - height[i]
```

The brute-force approach repeatedly finds both maximums → $O(N^2)$.

We can precompute leftMax and rightMax → $O(N)$ but use $O(N)$ space.

The key observation for the two-pointer solution:

If

```
leftMax < rightMax
```

then the water on the left side is determined by leftMax; we don't need the exact maximum on the right.

Similarly, if $\text{rightMax} < \text{leftMax}$, process the right side.

Therefore we can maintain:

```

leftMax
rightMax
left
right

```

and solve everything in one pass.

Brute — $O(N^2)$

Better — Prefix/Suffix — $O(N)$ time, $O(N)$ space

Optimal — $O(N)$ / $O(1)$

```
int trap(vector<int>& h)
{
    int l=0,r=h.size()-1;
    int leftMax=0,rightMax=0;
    int ans=0;

    while(l<=r)
    {
        if(h[l]<=h[r])
        {
            if(h[l]>=leftMax)
                leftMax=h[l];
            else
                ans+=leftMax-h[l];

            l++;
        }
        else
        {
            if(h[r]>=rightMax)
                rightMax=h[r];
            else
                ans+=rightMax-h[r];

            r--;
        }
    }

    return ans;
}
```

Key takeaway

When a problem depends on information from both left and right, ask whether two pointers can eliminate the need for extra arrays.

Chapter 2 — What You Should Remember

Don't memorize six codes. Memorize these six ideas:

1. Sorted + pair
↓
Two Pointers
2. In-place filtering
↓
Fast + Slow Pointer
3. Max area between two ends
↓
Move the shorter side
4. Three categories
↓
Three Pointers
5. Triplet
↓
Fix one + Two Sum

6. Left + Right dependency



Two Pointers + Maintain Max

☆ **Priority for your coding round**

If you have limited time, solve these first:

3Sum → Container With Most Water → Trapping Rain Water → Sort Colors → Two Sum II → Remove Element

These six give you the strongest coverage of the Two Pointer pattern.

DSA Array Mastery Handbook

Chapter 3 — Sliding Window

Table of Contents

- 🎯 Goal
- 🔑 Main Pattern: Sliding Window
- ★ Most Important 6 Problems
- Problem 1: Maximum Sum Subarray of Size K
- Problem 2: Maximum Average Subarray I
- Problem 3: Longest Substring Without Repeating Characters
- Problem 4: Longest Repeating Character Replacement
- Problem 5: Minimum Size Subarray Sum
- Problem 6: Max Consecutive Ones III
- 🌐 Chapter 3 — Pattern Recognition

📖 Chapter 3 — Sliding Window

🎯 Goal

Learn how to efficiently solve problems involving contiguous subarrays/substrings by maintaining a moving window instead of repeatedly recalculating the same elements.

🔑 Main Pattern: Sliding Window

The basic idea is:

```
[ L ----- R ]
```

Instead of calculating every subarray from scratch, maintain information about the current window.

When the window needs to change:

```
Expand → R++  
Shrink → L++
```

Two main types

1. Fixed-size window

Window size is always k.

```
[L ---- R]  
size k
```

Common in maximum/minimum sum problems.

2. Variable-size window

Window expands and shrinks depending on a condition.

```
[L ----- R]
```

```
↓ condition violated
[L ----- R]
```

Common in longest/shortest substring or subarray problems.

★ **Most Important 6 Problems**

For your handbook, I recommend these:

#	Problem	Pattern	Importance
1	Maximum Sum Subarray of Size K	Fixed Window	★★★★★
2	Maximum Average Subarray I	Fixed Window	★★★★
3	Longest Substring Without Repeating Characters	Variable Window + Hashing	★★★★★
4	Longest Repeating Character Replacement	Variable Window	★★★★★
5	Minimum Size Subarray Sum	Variable Window	★★★★★
6	Max Consecutive Ones III	Variable Window	★★★★★

These six cover the core sliding-window patterns.

Problem 1: Maximum Sum Subarray of Size K

Pattern: Fixed Sliding Window

Problem

Find the maximum sum of any contiguous subarray of size k.

```
nums = [2,1,5,1,3,2]
k = 3

Output = 9

[5,1,3] = 9
```

★ **Intuition**

Brute force calculates the sum of every window separately.

For example:

```
2+1+5
1+5+1
5+1+3
1+3+2
```

But notice something:

When moving from one window to the next, most elements remain the same.

For example:

```
[2,1,5] → [1,5,1]
```

We only:

```
Remove 2
Add 1
```

So instead of calculating the entire sum again, maintain a running sum.

This is the fundamental idea of a fixed-size sliding window.

Brute — $O(NK)$

```
int maxSum(vector<int>& a,int k)
{
    int ans=INT_MIN;

    for(int i=0;i+k<=a.size();i++)
    {
        int sum=0;

        for(int j=i;j<i+k;j++)
            sum+=a[j];

        ans=max(ans,sum);
    }

    return ans;
}
```

Optimal — $O(N)$

```
int maxSum(vector<int>& a,int k)
{
    int sum=0;

    for(int i=0;i<k;i++)
        sum+=a[i];

    int ans=sum;

    for(int i=k;i<a.size();i++)
    {
        sum+=a[i];
        sum-=a[i-k];

        ans=max(ans,sum);
    }

    return ans;
}
```

Space: $O(1)$

Key takeaway

Fixed window → Add the new element, remove the old element.

Problem 2: Maximum Average Subarray I

LeetCode: 643

Pattern: Fixed Sliding Window

Problem

Find the contiguous subarray of length k having the maximum average.

```
nums = [1,12,-5,-6,50,3]
k = 4
```

```
Output = 12.75
```

☆ Intuition

The denominator is always k , which is fixed.

Therefore:

Maximum average = maximum sum / k

So we only need to find the maximum sum of a window of size k.

This is exactly the fixed sliding-window pattern from Problem 1.

Brute

Calculate every window's sum.

Time: $O(NK)$

Optimal

Maintain a running window sum.

Time: $O(N)$

Space: $O(1)$

```
double findMaxAverage(vector<int>& a,int k)
{
    long long sum=0;

    for(int i=0;i<k;i++)
        sum+=a[i];

    long long best=sum;

    for(int i=k;i<a.size();i++)
    {
        sum+=a[i]-a[i-k];
        best=max(best,sum);
    }

    return (double)best/k;
}
```

Key takeaway

If the window size is fixed, think Sliding Window immediately.

Problem 3: Longest Substring Without Repeating Characters

LeetCode: 3

Pattern: Variable Sliding Window + Hashing

Problem

Find the length of the longest substring without repeating characters.

Input: "abcabcbb"

Output: 3

"abc"

★ Intuition

We need a longest valid substring.

Start with an empty window:

[L - - - - R]

Expand R while all characters are unique.

When a duplicate appears, the window becomes invalid.

Instead of starting over, move L forward until the duplicate is removed.

For example:

```
abc → valid

abca
  ↑ duplicate a
```

Move L

```
bca → valid
```

We maintain a set/map to know which characters are currently inside the window.

The important idea is:

Expand while valid; shrink when invalid.

This is the basic variable-size sliding window.

Brute — $O(N^2)$

Try every starting position and keep extending until a duplicate appears.

Better/Optimal — $O(N)$

Use a frequency array/map and two pointers.

```
int lengthOfLongestSubstring(string s)
{
    vector<int> freq(256,0);

    int l=0,ans=0;

    for(int r=0;r<s.size();r++)
    {
        freq[s[r]]++;

        while(freq[s[r]]>1)
        {
            freq[s[l]]--;
            l++;
        }

        ans=max(ans,r-l+1);
    }

    return ans;
}
```

Time: $O(N)$

Space: $O(1)$ for ASCII characters.

Key takeaway

Longest valid substring → expand → violation → shrink.

Problem 4: Longest Repeating Character Replacement

LeetCode: 424

Pattern: Variable Sliding Window

Problem

You can replace at most k characters.

Find the longest substring that can be converted into the same character.

```
s = "AABABBA"
k = 1

Output = 4
```

For example:

```
AABA
```

can become:

```
AAAA
```

with one replacement.

★ Intuition

Suppose our current window has:

```
A A B A
```

Frequency:

```
A = 3
B = 1
```

To make the entire window the same character, we need to replace everything except the most frequent character.

Therefore:

```
replacements = windowSize - maxFrequency
```

If:

```
windowSize - maxFrequency <= k
```

the window is valid.

Otherwise, shrink it from the left.

This gives us a powerful general sliding-window idea:

Window size – most frequent element = number of changes required.

Brute

Try every substring and calculate frequencies.

Time: approximately $O(N^2)$

Optimal — $O(N)$

```
int characterReplacement(string s,int k)
{
    vector<int> freq(26,0);

    int l=0,maxFreq=0,ans=0;

    for(int r=0;r<s.size();r++)
    {
        freq[s[r] - 'A']++;

        maxFreq=max(maxFreq,
                    freq[s[r] - 'A']);
```

```

        while((r-l+1)-maxFreq > k)
        {
            freq[s[l]-'A']--;
            l++;
        }

        ans=max(ans,r-l+1);
    }

    return ans;
}

```

Space: $O(1)$

Key takeaway

For replacement problems, ask: How many elements must I change to make this window valid?

Problem 5: Minimum Size Subarray Sum

LeetCode: 209

Pattern: Variable Sliding Window

Problem

Given an array of positive integers, find the minimum length subarray whose sum is at least target.

```

target = 7
nums = [2,3,1,2,4,3]

Output = 2

[4,3]

```

☆ Intuition

Because all numbers are positive, increasing the window always increases the sum, and shrinking it always decreases the sum.

So:

```

Expand the right pointer until  $\text{sum} \geq \text{target}$ .
Now the window is valid.
Try shrinking from the left as much as possible.
Record the smallest valid window.

```

Example:

```

[2,3,1,2,4]
sum = 12

shrink →

[3,1,2,4]
sum = 10

shrink →

[1,2,4]
sum = 7

shrink →

[2,4]
sum = 6 ✗

```

The smallest valid window was [1,2,4] of length 3 at this stage; later [4,3] gives length 2.

The crucial reason sliding window works here is the positive-number condition.

Brute — $O(N^2)$

```
int minSubArrayLen(int target, vector<int>& a)
{
    int ans=INT_MAX;

    for(int i=0;i<a.size();i++)
    {
        int sum=0;

        for(int j=i;j<a.size();j++)
        {
            sum+=a[j];

            if(sum>=target)
            {
                ans=min(ans,j-i+1);
                break;
            }
        }
    }

    return ans==INT_MAX ? 0 : ans;
}
```

Optimal — $O(N)$

```
int minSubArrayLen(int target, vector<int>& a)
{
    int l=0,sum=0,ans=INT_MAX;

    for(int r=0;r<a.size();r++)
    {
        sum+=a[r];

        while(sum>=target)
        {
            ans=min(ans,r-l+1);
            sum-=a[l++];
        }
    }

    return ans==INT_MAX ? 0 : ans;
}
```

Space: $O(1)$

Key takeaway

Positive numbers + minimum/maximum subarray satisfying a condition → Sliding Window is a strong candidate.

Problem 6: Max Consecutive Ones III

LeetCode: 1004

Pattern: Variable Sliding Window

Problem

Given a binary array, you can flip at most k zeroes.

Find the longest subarray containing only 1s after at most k flips.

```
nums = [1,1,1,0,0,0,1,1,1,1,0]
k = 2

Output = 6
```

☆ Intuition

We don't actually need to perform the flips.

Instead, consider a window.

The window is valid if it contains at most k zeroes.

```
zeroes <= k
```

Expand the right pointer.

If zeroes become greater than k , shrink from the left until the window becomes valid again.

The largest valid window is the answer.

This is almost the same pattern as Longest Repeating Character Replacement:

```
Maintain a window
    ↓
Count the "bad" elements
    ↓
If bad > k
    ↓
Shrink
    ↓
Keep maximum window
```

Brute — $O(N^2)$

Try every subarray and count zeroes.

Optimal — $O(N)$

```
int longestOnes(vector<int>& nums, int k)
{
    int l=0, zeros=0, ans=0;

    for(int r=0; r<nums.size(); r++)
    {
        if (nums[r]==0)
            zeros++;

        while(zeros>k)
        {
            if (nums[l]==0)
                zeros--;

            l++;
        }

        ans=max(ans, r-l+1);
    }

    return ans;
}
```

Space: $O(1)$

Key takeaway

At most K bad elements → maintain the count of bad elements inside the window.

Chapter 3 — Pattern Recognition

This is the most important part to remember.

Fixed Window

If you see:

```
"subarray/substring of exactly K size"
```

Think:

Fixed Sliding Window

Examples:

```
Maximum Sum of Size K  
Maximum Average of Size K
```

Variable Window — Longest

If you see:

```
"longest/maximum subarray or substring satisfying a condition"
```

Think:

Expand → violation → shrink → maximize

Examples:

```
Longest Substring Without Repeating Characters  
Longest Repeating Character Replacement  
Max Consecutive Ones III
```

Variable Window — Minimum

If you see:

```
"minimum/shortest subarray satisfying a condition"
```

Think:

```
Expand until valid  
    ↓  
Shrink as much as possible  
    ↓  
Minimize answer
```

Example:

```
Minimum Size Subarray Sum
```

★ Chapter 3 Priority

If you're short on time, prioritize these:

1. Longest Substring Without Repeating Characters ★★★★★
2. Minimum Size Subarray Sum ★★★★★
3. Longest Repeating Character Replacement ★★★★★
4. Max Consecutive Ones III ★★★★★
5. Maximum Sum Subarray of Size K ★★★★
6. Maximum Average Subarray I ★★★

One rule to remember:

Contiguous + window condition + need longest/shortest/count → Think Sliding Window first.

Important caveat

Standard sliding window for sum-based conditions usually relies on non-negative/positive numbers. With arbitrary negative numbers, the window may need a different technique such as prefix sum + hashing/deque.

DSA Array Mastery Handbook

Chapter 4 — Prefix Sum & Hashing

Table of Contents

- 🎯 Goal
- 🔑 Pattern 1: Prefix Sum
- 🔑 Pattern 2: Prefix Sum + Hashing
- 🔑 Pattern 3: Frequency Hashing
- ★ Most Important 6 Problems
- 1. Two Sum
- 2. Majority Element
- 3. Subarray Sum Equals K
- 4. Longest Subarray With Sum K
- 5. Longest Consecutive Sequence
- 6. Subarray Sum Divisible by K
- 🧠 Chapter 4 — Most Important Concepts
- 📄 Pattern Recognition Cheat Sheet

📄 Chapter 4 — Prefix Sum & Hashing

🎯 Goal

This chapter teaches two extremely powerful ideas:

Prefix Sum → avoid repeatedly calculating sums of subarrays.

Hashing → remember what we've already seen so we can find information in $O(1)$ average time.

Together, they solve many subarray, frequency, and counting problems.

🔑 Pattern 1: Prefix Sum

Suppose:

```
a = [2, 4, 1, 3, 5]
```

Define:

```
prefix[i] = sum of elements from 0 to i
```

So:

```
a:      2   4   1   3   5
prefix: 2   6   7  10  15
```

Now the sum from index L to R is:

```
sum(L,R) = prefix[R] - prefix[L-1]
```

For example:

```
sum(1,3) = 4 + 1 + 3
         = 8
```

Using prefix:

```
prefix[3] - prefix[0]
= 10 - 2
= 8
```

So a range-sum calculation becomes $O(1)$ after preprocessing.

🔑 Pattern 2: Prefix Sum + Hashing

This is one of the most important array patterns for interviews.

Suppose we want a subarray whose sum is K .

If current prefix sum is:

```
currentSum
```

and an earlier prefix sum was:

```
currentSum - K
```

then:

```
currentSum - (currentSum - K) = K
```

Therefore, a subarray with sum K exists between those two positions.

So we store previous prefix sums in a hash map.

Core idea

```
Current Prefix Sum = S
```

```
Need previous prefix sum:
```

```
S - K
```

This simple equation is worth remembering.

🔑 Pattern 3: Frequency Hashing

Sometimes the problem isn't about sums.

It asks:

```
How many times does something occur?
Have we seen this before?
Which element appears most?
Is there a duplicate?
Is the frequency balanced?
```

Then think:

```
unordered_map / unordered_set
```

★ Most Important 6 Problems

For your handbook, I recommend:

#	Problem	Main Pattern	Importance
1	Two Sum	Hashing	★★★★★
2	Majority Element	Hashing / Voting	★★★★★
3	Subarray Sum Equals K	Prefix Sum + Hashing	★★★★★
4	Longest Subarray with Sum K	Prefix Sum + Hashing	★★★★★
5	Longest Consecutive Sequence	Hashing	★★★★★
6	Subarray Sum Divisible by K	Prefix Sum + Hashing	★★★★

1. Two Sum

LeetCode: 1

Pattern: Hashing

Problem

Find two indices whose elements add up to target.

```
nums = [2,7,11,15]
target = 9
```

```
Output:
[0,1]
```

★ Intuition

Suppose the current number is x.

We need:

```
x + y = target
```

Therefore:

```
y = target - x
```

So while traversing the array, ask:

Have I already seen target - x?

If yes, we've found our pair.

This avoids checking every pair.

Example

```
target = 9

Current = 7

Need = 9 - 7 = 2

2 was already seen
    ↓
Pair found
```

Brute — $O(N^2)$

```
vector<int> twoSum(vector<int>& a,int target)
{
    for(int i=0;i<a.size();i++)
```

```

    {
        for(int j=i+1;j<a.size();j++)
        {
            if(a[i]+a[j]==target)
                return {i,j};
        }
    }

    return {};
}

```

Optimal — Hashing

Time: $O(N)$ average

Space: $O(N)$

```

vector<int> twoSum(vector<int>& a,int target)
{
    unordered_map<int,int> mp;

    for(int i=0;i<a.size();i++)
    {
        int need=target-a[i];

        if(mp.count(need))
            return {mp[need],i};

        mp[a[i]]=i;
    }

    return {};
}

```

🔑 Key Idea

Need something → calculate what is missing → look it up in a HashMap.

This is one of the most reusable hashing patterns.

2. Majority Element

LeetCode: 169

Pattern: Frequency / Boyer-Moore

Problem

Find the element that appears more than $N/2$ times.

[2,2,1,1,1,2,2]

Answer = 2

★ Intuition

The obvious solution is to count the frequency of every number.

But there's a stronger observation:

If an element occurs more than half the time, it cannot be completely cancelled by all other elements.

Imagine:

```

Majority: A A A A A
Others:   B C B C

```

Pair a majority element with a different element.

Because majority occurs more than $N/2$, some majority elements must remain.

This leads to Boyer-Moore Voting Algorithm.

Maintain:

```
candidate
count
```

If count becomes zero, choose the current element as the new candidate.

Brute — HashMap

```
int majorityElement(vector<int>& a)
{
    unordered_map<int,int> mp;

    for(int x:a)
        mp[x]++;

    for(auto [x,f]:mp)
        if(f>a.size()/2)
            return x;

    return -1;
}
```

Time: $O(N)$

Space: $O(N)$

Optimal — Boyer-Moore

Time: $O(N)$

Space: $O(1)$

```
int majorityElement(vector<int>& a)
{
    int candidate=0,count=0;

    for(int x:a)
    {
        if(count==0)
            candidate=x;

        if(x==candidate)
            count++;
        else
            count--;
    }

    return candidate;
}
```

🔑 Key Idea

A true majority can survive pairwise cancellation.

Interview Note

The standard LeetCode problem guarantees that a majority element exists. If the problem does not guarantee this, perform a second pass to verify the candidate's frequency.

3. Subarray Sum Equals K

LeetCode: 560

Pattern: Prefix Sum + HashMap

Problem

Count the number of subarrays whose sum equals K.

```
nums = [1,1,1]
k = 2
```

Output = 2

Subarrays:

```
[1,1]
[1,1]
```

★ Intuition

This is one of the most important prefix-sum problems.

Suppose current prefix sum is:

```
S
```

We want a previous prefix sum P such that:

$$S - P = K$$

Therefore:

$$P = S - K$$

So at every index:

Check whether $\text{prefixSum} - K$ has appeared before.

If it has appeared x times, then there are x subarrays ending at the current position having sum K.

That's why we store frequency, not just existence.

Brute — $O(N^2)$

```
int subarraySum(vector<int>& a,int k)
{
    int ans=0;

    for(int i=0;i<a.size();i++)
    {
        int sum=0;

        for(int j=i;j<a.size();j++)
        {
            sum+=a[j];

            if(sum==k)
                ans++;
        }
    }

    return ans;
}
```

Optimal — Prefix Sum + HashMap

Time: $O(N)$ average

Space: $O(N)$

```
int subarraySum(vector<int>& a,int k)
{
    unordered_map<int,int> mp;

    mp[0]=1;

    int sum=0,ans=0;
```

```

for(int x:a)
{
    sum+=x;

    if(mp.count(sum-k))
        ans+=mp[sum-k];

    mp[sum]++;
}

return ans;
}

```

Why mp[0] = 1?

Consider:

```

[3]
k = 3

```

Current prefix sum = 3.

We need:

```

3 - 3 = 0

```

So prefix sum 0 must already exist.

It represents a subarray starting from index 0.

🔑 Key Formula

Current Prefix Sum = S

Required Previous Prefix Sum = S - K

Memorize this pattern.

4. Longest Subarray With Sum K

Pattern: Prefix Sum + HashMap

Problem

Find the length of the longest subarray whose sum equals K.

```

nums = [10,5,2,7,1,9]
k = 15

Output = 4

[5,2,7,1]

```

★ Intuition

The same prefix-sum equation applies:

```

currentSum - previousSum = K

```

Therefore:

```

previousSum = currentSum - K

```

But this problem asks for the longest subarray.

So when a prefix sum appears for the first time, store its index.

Why the first occurrence?

Suppose:

```
prefix sum = 5
```

appears at index 2 and again at index 7.

If we need that prefix sum later, using index 2 gives a longer subarray than using index 7.

Therefore:

For longest subarray, store the first occurrence of each prefix sum.

Brute — $O(N^2)$

```
int longestSubarray(vector<int>& a,int k)
{
    int ans=0;

    for(int i=0;i<a.size();i++)
    {
        int sum=0;

        for(int j=i;j<a.size();j++)
        {
            sum+=a[j];

            if(sum==k)
                ans=max(ans,j-i+1);
        }
    }

    return ans;
}
```

Optimal — $O(N)$

```
int longestSubarray(vector<int>& a,int k)
{
    unordered_map<long long,int> mp;

    long long sum=0;
    int ans=0;

    for(int i=0;i<a.size();i++)
    {
        sum+=a[i];

        if(sum==k)
            ans=i+1;

        if(mp.count(sum-k))
            ans=max(ans,i-mp[sum-k]);

        if(!mp.count(sum))
            mp[sum]=i;
    }

    return ans;
}
```

🧠 Important Difference

For counting subarrays:

```
Store frequency
```

For longest subarray:

```
Store first index
```

This distinction is extremely important.

5. Longest Consecutive Sequence

LeetCode: 128

Pattern: HashSet

Problem

Find the length of the longest consecutive sequence.

```
[100,4,200,1,3,2]
```

```
Output = 4
```

```
Sequence:
```

```
1,2,3,4
```

★ Intuition

Sorting gives an easy solution, but we can do better.

Put every number into a HashSet.

Now for each number x , ask:

```
Is x-1 present?
```

If yes, x is not the beginning of a sequence.

If no, then x is a sequence starting point.

For example:

```
1 → 0 doesn't exist
    ↓
sequence starts here
```

```
2 → 1 exists
3 → 2 exists
4 → 3 exists
```

Then keep moving forward:

```
1 → 2 → 3 → 4
```

This prevents unnecessary work.

Brute — $O(N^2)$

For every element, repeatedly search for the next consecutive number.

Better — Sorting

Time: $O(N \log N)$

```
sort(nums.begin(), nums.end());
```

Then scan consecutive values.

Optimal — HashSet

Average Time: $O(N)$

Space: $O(N)$

```
int longestConsecutive(vector<int>& nums)
{
    unordered_set<int> st(nums.begin(),nums.end());

    int ans=0;

    for(int x:st)
    {
        if(!st.count(x-1))
        {
            int cur=x;
            int len=1;

            while(st.count(cur+1))
            {
                cur++;
                len++;
            }

            ans=max(ans,len);
        }
    }

    return ans;
}
```

🔑 Key Idea

Don't start a sequence from every number. Start only when $x-1$ doesn't exist.

6. Subarray Sum Divisible by K

LeetCode: 974

Pattern: Prefix Sum + Remainder Hashing

Problem

Count the number of subarrays whose sum is divisible by K.

```
nums = [4,5,0,-2,-3,1]
k = 5

Output = 7
```

★ Intuition

This is a beautiful extension of prefix sum.

Suppose two prefix sums have the same remainder when divided by K.

For example:

```
Prefix A % K = 2
Prefix B % K = 2
```

Then:

```
(PrefixB - PrefixA) % K = 0
```

Therefore, the subarray between them has a sum divisible by K.

So instead of storing the actual prefix sums, store:

```
prefixSum % K
```

At every position:

If this remainder has appeared before, every previous occurrence forms a valid subarray ending here.

Brute — $O(N^2)$

Calculate every subarray sum and check:

```
sum % k == 0
```

Optimal — $O(N)$

```
int subarraysDivByK(vector<int>& a,int k)
{
    unordered_map<int,int> mp;

    mp[0]=1;

    int sum=0,ans=0;

    for(int x:a)
    {
        sum+=x;

        int rem=sum%k;

        if(rem<0)
            rem+=k;

        if(mp.count(rem))
            ans+=mp[rem];

        mp[rem]++;
    }

    return ans;
}
```

Why $rem < 0$?

In C++, negative numbers can produce negative remainders.

For example:

```
-2 % 5 = -2
```

But we want the remainder represented as:

```
3
```

So:

```
if(rem < 0)
    rem += k;
```

🔑 Key Idea

Same prefix remainder → subarray between them is divisible by K.

🔑 Chapter 4 — Most Important Concepts

1. Prefix Sum

Remember:

```
sum(L,R) = prefix[R] - prefix[L-1]
```

Useful when:

```
Repeated range sums
Subarray sums
Range queries
```

2. Prefix Sum + HashMap

The most important formula:

```
previousPrefix = currentPrefix - K
```

Used in:

```
Subarray Sum Equals K
Longest Subarray Sum K
```

3. Frequency vs First Index

This is extremely important.

If asked:

```
"How many subarrays?"
```

Store:

```
mp[prefix]++;
```

If asked:

```
"Longest subarray?"
```

Store:

```
if(!mp.count(prefix))
    mp[prefix]=index;
```

4. Prefix Remainder

If asked:

```
Number of subarrays whose sum is divisible by K
```

Think:

```
prefix % K
```

Then:

```
same remainder → valid subarray
```

 Pattern Recognition Cheat Sheet

```
Need pair with target?
↓
Hashing
target - current

Need subarray sum = K?
↓
```


Prefix Sum + HashMap

prefix - K

Need longest subarray sum = K?

↓

Prefix Sum + first index

Need count of subarrays sum = K?

↓

Prefix Sum + frequency

Need sum divisible by K?

↓

Prefix Sum % K

Need longest consecutive sequence?

↓

HashSet

DSA Array Mastery Handbook

Chapter 5 — Kadane's Algorithm & Array Rearrangement

Table of Contents

- 🎯 Goal
- 🔑 Main Patterns
- ★ Most Important 6 Problems
- 1. Maximum Subarray
- 2. Maximum Product Subarray
- 3. Best Time to Buy and Sell Stock
- 4. Rearrange Array Elements by Sign
- 5. Next Permutation
- 6. Merge Intervals
- 🧠 Chapter 5 — Pattern Recognition
- 🕒 Chapter 5 — Priority
- 👉 Most Important Interview Insights

Chapter 5 — Kadane's Algorithm & Array Rearrangement

🎯 Goal

This chapter focuses on two important patterns:

Kadane's Algorithm → maximum/minimum sum of a contiguous subarray.

Array Rearrangement → intelligently placing elements according to a required pattern or condition.

🔑 Main Patterns

Pattern 1 — Kadane's Algorithm

The central question is:

Should I continue my current subarray, or should I start a new subarray from the current element?

For every element:

```
currentSum = max(current element,
                  currentSum + current element)
```

Intuition

Suppose your current subarray has a negative contribution.

If keeping that previous sum makes the current element worse, there is no reason to carry it forward.

So we decide:

Continue previous subarray

```
        OR
    Start new subarray
```

This reduces a potentially $O(N^2)$ problem to $O(N)$.

Pattern 2 — Rearrangement

Instead of simply sorting, ask:

```
What property must the final array satisfy?
```

Typical ideas include:

```
separate positive and negative numbers
alternate elements
move elements to specific positions
partition based on a condition
rearrange in-place
```

Often the solution uses:

```
Two pointers
Sorting
Greedy placement
In-place swapping
```

★ Most Important 6 Problems

For this chapter, I recommend:

#	Problem	Main Pattern	Importance
1	Maximum Subarray	Kadane	★★★★★
2	Maximum Product Subarray	Modified Kadane	★★★★★
3	Best Time to Buy and Sell Stock	Kadane-style	★★★★★
4	Rearrange Array Elements by Sign	Rearrangement	★★★★
5	Next Permutation	Rearrangement	★★★★★
6	Merge Intervals	Sorting + Rearrangement	★★★★★

1. Maximum Subarray

LeetCode: 53

Problem

Find the contiguous subarray having the maximum sum.

```
[-2,1,-3,4,-1,2,1,-5,4]

Answer = 6

[4,-1,2,1]
```

★ Main Pattern: Kadane's Algorithm

Brute Force Intuition

Try every possible starting point.

For each starting point, extend the subarray and keep calculating its sum.

```
Start at 0
→ [0]
→ [0...1]
→ [0...2]
...

Start at 1
→ [1]
→ [1...2]
...
```

We keep the maximum sum found.

Brute — $O(N^2)$

```
int maxSubArray(vector<int>& a)
{
    int ans=INT_MIN;

    for(int i=0;i<a.size();i++)
    {
        int sum=0;

        for(int j=i;j<a.size();j++)
        {
            sum+=a[j];
            ans=max(ans,sum);
        }
    }

    return ans;
}
```

Why is it better than $O(N^3)$?

We don't recalculate the sum of every subarray from scratch.

For a fixed i:

```
sum += a[j]
```

So each starting point takes $O(N)$.

★ Optimal Intuition — Kadane

Suppose:

```
currentSum = -5
current element = 4
```

Would you rather take:

```
-5 + 4 = -1
```

or simply:

```
4
```

Obviously 4.

So at every position:

Either:

```
Start new → a[i]
```

OR

```
Continue → currentSum + a[i]
```

Choose the larger one.

Optimal — $O(N)$

```
int maxSubArray(vector<int>& a)
{
    int curr=a[0];
    int ans=a[0];

    for(int i=1;i<a.size();i++)
    {
        curr=max(a[i],curr+a[i]);
        ans=max(ans,curr);
    }

    return ans;
}
```

Time: $O(N)$

Space: $O(1)$

🧠 Core Insight

If the previous sum hurts the current element, discard it.

2. Maximum Product Subarray

LeetCode: 152

Problem

Find the contiguous subarray having the maximum product.

```
[2,3,-2,4]
```

```
Answer = 6
```

```
[2,3]
```

This looks similar to Maximum Subarray, but there is an important difference.

★ Why normal Kadane doesn't work

For sums:

```
negative + positive
```

usually makes things smaller.

But for products:

```
negative × negative = positive
```

So a very small negative product can become the largest positive product later.

Example:

```
[-2,3,-4]
```

```
-2
```

```
-2 × 3 = -6
```

```
-6 × -4 = 24 ← maximum
```

Therefore, we need to remember both:

```
maximum product
minimum product
```

The minimum can suddenly become maximum after multiplying by a negative number.

Brute Force Intuition

Generate every subarray and maintain its product.

```
int maxProduct(vector<int>& a)
{
    int ans=INT_MIN;

    for(int i=0;i<a.size();i++)
    {
        int product=1;

        for(int j=i;j<a.size();j++)
        {
            product*=a[j];
            ans=max(ans,product);
        }
    }

    return ans;
}
```

Time: $O(N^2)$

★ Optimal Intuition

For each element, there are three possibilities:

```
current element
current × previous maximum
current × previous minimum
```

So maintain:

```
maxProduct
minProduct
```

When the current element is negative, the maximum and minimum effectively switch roles.

Optimal — $O(N)$

```
int maxProduct(vector<int>& a)
{
    int maxi=a[0];
    int mini=a[0];
    int ans=a[0];

    for(int i=1;i<a.size();i++)
    {
        if(a[i]<0)
            swap(maxi,mini);

        maxi=max(a[i],maxi*a[i]);
        mini=min(a[i],mini*a[i]);

        ans=max(ans,maxi);
    }

    return ans;
}
```

Space: $O(1)$

🧠 Core Insight

For products, track both maximum and minimum because a negative can flip their roles.

3. Best Time to Buy and Sell Stock

LeetCode: 121

Problem

You can buy once and sell once. Find the maximum profit.

```
prices = [7,1,5,3,6,4]
```

```
Answer = 5
```

```
Buy at 1
```

```
Sell at 6
```

★ Main Pattern

This can be viewed as a Kadane-style / one-pass optimization problem.

Brute Force Intuition

Try every possible buying day and every later selling day.

```
profit = prices[j] - prices[i]
```

Keep the maximum.

Brute — $O(N^2)$

```
int maxProfit(vector<int>& prices)
{
    int ans=0;

    for(int i=0;i<prices.size();i++)
    {
        for(int j=i+1;j<prices.size();j++)
        {
            ans=max(ans,prices[j]-prices[i]);
        }
    }

    return ans;
}
```

★ Optimal Intuition

When we are at today's price, ask:

```
What is the cheapest price I could have bought at before today?
```

Maintain:

```
minimum price seen so far
```

Then:

```
profit = current price - minimum price
```

Every day we update the best profit.

Optimal — $O(N)$

```
int maxProfit(vector<int>& prices)
{
    int minPrice=prices[0];
    int profit=0;

    for(int i=1;i<prices.size();i++)
    {
        profit=max(profit,prices[i]-minPrice);
        minPrice=min(minPrice,prices[i]);
    }

    return profit;
}
```

Space: $O(1)$

Core Insight

For every selling price, keep the cheapest buying price seen before it.

4. Rearrange Array Elements by Sign

LeetCode: 2149

Problem

Given equal numbers of positive and negative elements, rearrange them alternately.

Input:
[3,1,-2,-5,2,-4]

Output:
[3,-2,1,-5,2,-4]

Positive and negative elements should alternate.

☆ Brute Intuition

Separate the numbers into two arrays:

positive → [3,1,2]
negative → [-2,-5,-4]

Then place them alternately.

This is straightforward but uses extra space.

Brute — $O(N)$ / $O(N)$

```
vector<int> rearrangeArray(vector<int>& a)
{
    vector<int> pos,neg;

    for(int x:a)
    {
        if(x>0) pos.push_back(x);
        else neg.push_back(x);
    }

    vector<int> ans;

    for(int i=0;i<pos.size();i++)
    {
        ans.push_back(pos[i]);
    }
}
```



```

        ans.push_back(neg[i]);
    }

    return ans;
}

```

★ Optimal Intuition

We already know where each type belongs.

If the answer starts with positive:

```

positive positions:
0,2,4,...

negative positions:
1,3,5,...

```

So we can maintain two indices:

```

posIndex = 0
negIndex = 1

```

and place elements directly.

Optimal — $O(N)$

```

vector<int> rearrangeArray(vector<int>& a)
{
    vector<int> ans(a.size());

    int pos=0,neg=1;

    for(int x:a)
    {
        if(x>0)
        {
            ans[pos]=x;
            pos+=2;
        }
        else
        {
            ans[neg]=x;
            neg+=2;
        }
    }

    return ans;
}

```

Time: $O(N)$

Space: $O(N)$ for the required output array.

🧠 Core Insight

When the final positions follow a fixed pattern, directly maintain those positions.

5. Next Permutation

LeetCode: 31

Problem

Find the next lexicographically greater permutation.

```
[1,2,3]
```

```
→ [1,3,2]
```

For:

```
[3,2,1]
```

the next permutation is:

```
[1,2,3]
```

because [3,2,1] is already the largest permutation.

★ Brute Intuition

Generate all permutations, sort them, and find the current permutation's next one.

This is extremely expensive because there are:

```
N!
```

permutations.

So this approach is mainly useful for understanding, not for interviews.

★ Optimal Intuition

Look from the right.

We want the smallest possible increase.

Step 1

Find the first index from the right where:

```
a[i] < a[i+1]
```

This is called the pivot.

Example:

```
1 2 5 4 3
    ↑
  pivot

2 < 5.
```

Step 2

Find the smallest element from the right that is greater than the pivot.

```
1 2 5 4 3
    ↑
  pivot = 2

Choose 3
```

Swap them:

```
1 3 5 4 2
```

Step 3

Reverse everything after the pivot:

1 3 2 4 5

This gives the smallest permutation greater than the original.

Optimal — $O(N)$

```
void nextPermutation(vector<int>& a)
{
    int n=a.size();

    int i=n-2;

    while(i>=0 && a[i]>=a[i+1])
        i--;

    if(i>=0)
    {
        int j=n-1;

        while(a[j]<=a[i])
            j--;

        swap(a[i],a[j]);
    }

    reverse(a.begin()+i+1,a.end());
}
```

Time: $O(N)$

Space: $O(1)$

🧠 Core Insight

Find the rightmost place where you can make the permutation slightly larger, then make the suffix as small as possible.

This is a very important rearrangement pattern.

6. Merge Intervals

LeetCode: 56

Although this is technically an interval problem, it is extremely important for array/interview preparation because it teaches sorting + merging/rearrangement.

Problem

Merge overlapping intervals.

```
[1,3]
[2,6]
[8,10]
[9,11]
```

Output:

```
[1,6]
[8,11]
```

★ Brute Intuition

Compare every interval with every other interval and repeatedly merge overlapping intervals.

The difficulty is that after merging, new overlaps can appear.

This makes the implementation complicated and can lead to roughly $O(N^2)$ behavior.

★ Optimal Intuition

Sort intervals by their starting point.

Then the intervals have a useful property:

```
[1,3]
[2,6]
[8,10]
[9,11]
```

Now we only need to compare the current interval with the last merged interval.

If:

```
current.start <= last.end
```

they overlap.

So extend:

```
last.end = max(last.end, current.end)
```

Otherwise, start a new interval.

Optimal — $O(N \log N)$

Sorting dominates the complexity.

```
vector<vector<int>> merge(vector<vector<int>>& a)
{
    sort(a.begin(), a.end());

    vector<vector<int>> ans;

    for(auto interval:a)
    {
        if(ans.empty() ||
           ans.back()[1] < interval[0])
        {
            ans.push_back(interval);
        }
        else
        {
            ans.back()[1] = max(ans.back()[1],
                               interval[1]);
        }
    }

    return ans;
}
```

Time: $O(N \log N)$

Space: $O(N)$ for the output.

🧠 Core Insight

Sort first → overlapping intervals become neighbors → merge with the last interval.

🧠 Chapter 5 — Pattern Recognition

This is what you should actually remember instead of memorizing the code.

1. Maximum Sum

```
Should I continue?
↓
max(current, current + x)
```

↓
Kadane

2. Maximum Product

Negative can become positive
↓
Track MAX + MIN

3. Stock Profit

Current selling price
-
Minimum price before it
↓
Maximum profit

4. Rearrangement

Does every element have
a predetermined position?
↓
Maintain position pointers

5. Next Permutation

Find pivot
↓
Find slightly larger element
↓
Swap
↓
Reverse suffix

6. Merge Intervals

Sort by start
↓
Compare with last merged
↓
Overlap → merge
No overlap → new interval

Chapter 5 — Priority

If you're short on time:

Must Master ★★★★★

Maximum Subarray — Kadane
Maximum Product Subarray
Best Time to Buy/Sell Stock
Next Permutation
Merge Intervals

Then

Rearrange Array by Sign

Most Important Interview Insights

Keep these six lines in your notes:

Kadane:

"If previous sum hurts me, start fresh."

Maximum Product:

"Negative can turn minimum into maximum."

Stock:

"Sell today → buy at the minimum price seen before."

Rearrangement:

"Know the required positions and fill them directly."

Next Permutation:

"Make the smallest possible increase."

Merge Intervals:

"Sort first, then merge neighboring overlaps."

Chapter 6 — Matrix

Goal

Matrix problems are basically 2D array problems. The main skill is not memorizing formulas, but learning how to control:

- rows and columns
- boundaries
- directions
- in-place transformations
- traversal patterns

For your handbook, I recommend 6 important problems.

Main Patterns in Matrix Problems

There are 5 patterns you should recognize:

1. Row/Column Traversal

```
for each row
  for each column
```

Used for searching, counting, or modifying cells.

2. Boundary Traversal

Instead of visiting every cell randomly, process:

```
Top → Right → Bottom → Left
```

This is the foundation of Spiral Matrix.

3. In-place Matrix Transformation

Instead of creating another matrix, modify the existing matrix.

Examples:

- Rotate Matrix
- Set Matrix Zeroes

4. Transpose + Reverse

A very useful trick:

Rotate a square matrix 90° clockwise = Transpose + Reverse every row

5. Matrix Search Using Sorted Properties

If rows/columns are sorted, don't blindly use $O(MN)$.

Use the structure of the matrix to eliminate rows/columns.

★ Most Important 6 Problems

#	Problem	Main Pattern	Importance
1	Set Matrix Zeroes	Row/Column marking	★★★★★
2	Rotate Matrix 90°	Transpose + Reverse	★★★★★
3	Spiral Matrix	Boundary traversal	★★★★★
4	Search a 2D Matrix	Binary Search	★★★★★
5	Search in Row/Column Sorted Matrix	Staircase Search	★★★★★
6	Pascal's Triangle	Combinational construction	★★★★★

1. Set Matrix Zeroes

LeetCode: 73

Problem

If an element is 0, set its entire row and column to 0.

Input:

```
1 1 1
1 0 1
1 1 1
```

Output:

```
1 0 1
0 0 0
1 0 1
```

★ Brute Force Intuition

Whenever we find a zero, we need to mark its entire row and column.

But there's a problem:

If we immediately change them to zero, those newly-created zeroes may be mistaken for original zeroes.

For example:

```
1 1 1
1 0 1
1 1 1
```

If we start changing cells immediately, we may create extra zeroes and incorrectly process them.

So we can use two arrays:

- `row[i] = 1` → row must become zero
- `col[j] = 1` → column must become zero

First mark all affected rows/columns, then perform the changes.

Brute — $O(MN)$ time, $O(M+N)$ space

```
void setZeroes(vector<vector<int>>& a)
{
    int m=a.size(), n=a[0].size();

    vector<int> row(m,0), col(n,0);

    for(int i=0;i<m;i++)
        for(int j=0;j<n;j++)
            if(a[i][j]==0)
            {
                row[i]=1;
                col[j]=1;
            }

    for(int i=0;i<m;i++)
        for(int j=0;j<n;j++)
            if(row[i] || col[j])
                a[i][j]=0;
}
```

★ Optimal Intuition

Can we avoid the extra row and column arrays?

Yes.

Use the first row and first column themselves as markers.

For example:

```
matrix[i][0] = 0
```

means:

Row i needs to become zero.

And:

```
matrix[0][j] = 0
```

means:

Column j needs to become zero.

But now there is a problem:

What if the original first row/first column already contains zeroes?

So maintain two extra variables:

- `firstRowZero`
- `firstColZero`

Then use the first row/column as markers.

Optimal — $O(MN)$ / $O(1)$ extra space

```

void setZeroes(vector<vector<int>>& a)
{
    int m=a.size(), n=a[0].size();

    bool firstRow=false, firstCol=false;

    for(int j=0;j<n;j++)
        if(a[0][j]==0)
            firstRow=true;

    for(int i=0;i<m;i++)
        if(a[i][0]==0)
            firstCol=true;

    for(int i=1;i<m;i++)
        for(int j=1;j<n;j++)
            if(a[i][j]==0)
            {
                a[i][0]=0;
                a[0][j]=0;
            }

    for(int i=1;i<m;i++)
        for(int j=1;j<n;j++)
            if(a[i][0]==0 || a[0][j]==0)
                a[i][j]=0;

    if(firstRow)
        for(int j=0;j<n;j++)
            a[0][j]=0;

    if(firstCol)
        for(int i=0;i<m;i++)
            a[i][0]=0;
}

```

Core Insight

Use the matrix itself as storage for row/column information.

2. Rotate Matrix 90° Clockwise

LeetCode: 48

Problem

Rotate an $N \times N$ matrix 90° clockwise in-place.

```

1 2 3
4 5 6
7 8 9

```

becomes:

```

7 4 1
8 5 2
9 6 3

```

★ Brute Force Intuition

Create another matrix.

The element at:

```
[i][j]
```

moves to:

```
[j][n-1-i]
```

So we can create a new matrix and put every element in its rotated position.

Time: $O(N^2)$ Space: $O(N^2)$

```
vector<vector<int>> rotateBrute(vector<vector<int>>& a)
{
    int n=a.size();

    vector<vector<int>> b(n,vector<int>(n));

    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            b[j][n-1-i]=a[i][j];

    return b;
}
```

★ Optimal Intuition

There is a beautiful observation:

```
90° clockwise rotation
Transpose
↓
Reverse every row
```

Example:

```
1 2 3
4 5 6
7 8 9
```

Step 1 — Transpose

```
1 4 7
2 5 8
3 6 9
```

Step 2 — Reverse each row

```
7 4 1
8 5 2
9 6 3
```

Done.

Optimal — $O(N^2)$ / $O(1)$

```
void rotate(vector<vector<int>>& a)
{
    int n=a.size();

    // Transpose
    for(int i=0;i<n;i++)
```

```

        for(int j=i+1;j<n;j++)
            swap(a[i][j],a[j][i]);

    // Reverse each row
    for(int i=0;i<n;i++)
        reverse(a[i].begin(),a[i].end());
}

```

Core Insight

Clockwise 90° = Transpose + Reverse rows.

This is one of the most important matrix tricks.

3. Spiral Matrix

LeetCode: 54

Problem

Return all elements in spiral order.

```

1 2 3
4 5 6
7 8 9

```

Output:

```

1 2 3 6 9 8 7 4 5

```

★ Brute Force Intuition

You could maintain a visited matrix and move:

```

right → down → left → up

```

Whenever you reach a boundary or visited cell, change direction.

This works, but requires $O(MN)$ extra space.

★ Optimal Intuition

Instead of tracking visited cells, track the four boundaries:

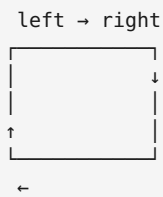
- top
- bottom
- left
- right

Process:

1. left → right
2. top → bottom
3. right → left
4. bottom → top

Then shrink the corresponding boundary.

Visualize:



Important

After processing the top row, increase:

```
top++;
```

After processing the right column:

```
right--;
```

and so on.

Optimal — $O(MN)$ / $O(1)$ extra space

```
vector<int> spiralOrder(vector<vector<int>>& a)
{
    vector<int> ans;

    int top=0;
    int bottom=a.size()-1;
    int left=0;
    int right=a[0].size()-1;

    while(top<=bottom && left<=right)
    {
        for(int j=left;j<=right;j++)
            ans.push_back(a[top][j]);

        top++;

        for(int i=top;i<=bottom;i++)
            ans.push_back(a[i][right]);

        right--;

        if(top<=bottom)
        {
            for(int j=right;j>=left;j--)
                ans.push_back(a[bottom][j]);

            bottom--;
        }

        if(left<=right)
        {
            for(int i=bottom;i>=top;i--)
                ans.push_back(a[i][left]);

            left++;
        }
    }

    return ans;
}
```

Core Insight

Spiral traversal = four boundaries + shrink after each direction.

4. Search a 2D Matrix

LeetCode: 74

Problem

Each row is sorted and the first element of every row is greater than the last element of the previous row.

Example:

```
1  3  5  7
10 11 16 20
23 30 34 60
```

Search for 16.

★ Brute Force Intuition

Simply visit every element.

```
for(...)
  for(...)
    if(matrix[i][j]==target)
```

Time: $O(MN)$

Simple, but doesn't use the sorted structure.

★ Better Intuition

First find the correct row using binary search.

Then perform another binary search inside that row.

```
Find row → Binary Search
           ↓
        Search row
```

Time: $O(\log M + \log N)$

★ Optimal Intuition

Think of the entire matrix as one sorted 1D array.

For:

```
3 × 4 matrix
```

we conceptually have:

```
1 3 5 7 10 11 16 20 23 30 34 60
```

For a 1D index mid:

```
row = mid / n
col = mid % n
```

So ordinary binary search works.

Optimal — $O(\log(MN))$

```
bool searchMatrix(vector<vector<int>>& a,int target)
{
    int m=a.size(),n=a[0].size();

    int l=0,r=m*n-1;

    while(l<=r)
    {
        int mid=l+(r-l)/2;

        int row=mid/n;
        int col=mid%n;

        if(a[row][col]==target)
            return true;

        if(a[row][col]<target)
            l=mid+1;
        else
            r=mid-1;
    }

    return false;
}
```

Core Insight

A properly structured sorted matrix can be treated as a sorted 1D array.

5. Search in a Row & Column Sorted Matrix

LeetCode: 240

Problem

Every row and every column is sorted.

```
1  4  7 11
2  5  8 12
3  6  9 16
10 13 14 17
```

Search for 5.

★ Brute Intuition

Search every cell.

Time: $O(MN)$

★ Better Intuition

Binary search each row.

For each row:

$O(\log N)$

Total:

$O(M \log N)$

But we can do even better.

★ Optimal Intuition — Staircase Search

Start at the top-right corner.

```
1  4  7 11
2  5  8 12
3  6  9 16
10 13 14 17
      ↑
    Start
```

At this position:

If current > target

Move left.

Why?

Everything below is even larger.

If current < target

Move down.

Why?

Everything to the left is smaller.

Therefore every move eliminates an entire row or column.

Optimal — $O(M + N)$

```
bool searchMatrix(vector<vector<int>>& a,int target)
{
    int m=a.size();
    int n=a[0].size();

    int i=0,j=n-1;

    while(i<m && j>=0)
    {
        if(a[i][j]==target)
            return true;

        if(a[i][j]>target)
            j--;
        else
            i++;
    }
}
```



```
    return false;
}
```

Core Insight

Top-right corner gives two directions of information.

This is called staircase search.

6. Pascal's Triangle

LeetCode: 118

Problem

Generate the first numRows of Pascal's Triangle.

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

★ Brute Intuition

One option is to calculate every value using combinations:

$$C(n, r) = n! / (r!(n-r)!)$$

But calculating factorials repeatedly is unnecessary.

★ Optimal Intuition

Every interior element is simply:

upper-left + upper-right

For example:

```
1   3   3   1
  \ / \ /
   6   4
```

The first and last values are always 1.

So:

$$\text{current}[j] = \text{previous}[j-1] + \text{previous}[j]$$

Optimal — $O(N^2)$

```
vector<vector<int>> generate(int n)
{
    vector<vector<int>> ans;

    for(int i=0; i<n; i++)
    {
```

```

        vector<int> row(i+1,1);

        for(int j=1;j<i;j++)
            row[j]=ans[i-1][j-1]+ans[i-1][j];

        ans.push_back(row);
    }

    return ans;
}

```

Core Insight

Each interior value comes from the two values directly above it.

Chapter 6 — Pattern Recognition

Instead of memorizing six solutions, remember these patterns:

Set Matrix Zeroes

```

Find zero
↓
Mark row + column
↓
Apply changes

```

Optimal:

Use first row/column as markers.

Rotate Matrix

```

90° clockwise
↓
Transpose
↓
Reverse rows

```

Spiral Matrix

```

Top
Right
Bottom
Left
↓
Shrink boundaries

```

Sorted Matrix + Search

If the matrix behaves like one sorted array:

```

Binary Search

```

Rows + Columns independently sorted

Start:

```

Top-right

```

Then:

Too large → left
Too small → down

Pascal's Triangle

Upper-left + Upper-right

Chapter 6 Priority

If you're short on time:

Must Master ★★★★★

- Set Matrix Zeroes
- Rotate Matrix
- Spiral Matrix
- Search a 2D Matrix
- Search in Row/Column Sorted Matrix

Then

- Pascal's Triangle

Interview Cheat Sheet

```
Matrix modification
  ↓
Think in-place marking

90° rotation
  ↓
Transpose + Reverse

Spiral
  ↓
4 boundaries

Sorted like 1D
  ↓
Binary Search

Rows + columns sorted
  ↓
Staircase Search

Pascal
  ↓
Upper-left + Upper-right
```

The 3 matrix patterns I'd especially memorize

1. Transpose + Reverse → Rotation
2. Four boundaries → Spiral
3. Top-right → Staircase Search

Chapter 7 — Binary Search on Arrays

Goal

Binary Search is not just:

| “Find an element in a sorted array.”

The bigger pattern is:

Use a condition that changes from false → true (or true → false), and eliminate half of the search space each time.

This chapter is extremely important because many array problems hide binary search behind words like:

- minimum possible
- maximum possible
- first/last occurrence
- rotated sorted array
- smallest valid answer
- capacity/speed/days

Main Patterns

Pattern 1 — Classic Binary Search

```
Sorted array
  ↓
Check middle
  ↓
Eliminate half
```

Pattern 2 — First/Last Occurrence

Instead of stopping when you find the target:

Continue searching to find the boundary.

Pattern 3 — Rotated Sorted Array

Even though the whole array isn't sorted, at least one half is always sorted.

Use that sorted half to decide where to search.

Pattern 4 — Binary Search on Answer

The array itself may not even be sorted.

Instead, the answer space is sorted:

```
Impossible Impossible Possible Possible
```

Then binary search the answer.

This is one of the most important advanced binary-search patterns.

★ Most Important 6 Problems

#	Problem	Main Pattern	Importance
1	Binary Search	Basic BS	★★★★★
2	First and Last Position	Boundary Search	★★★★★
3	Search in Rotated Sorted Array	Rotated BS	★★★★★
4	Find Minimum in Rotated Sorted Array	Rotated BS	★★★★★
5	Koko Eating Bananas	Binary Search on Answer	★★★★★
6	Capacity to Ship Packages Within D Days	Binary Search on Answer	★★★★★

1. Binary Search

LeetCode: 704

Problem

Find the index of target in a sorted array.

```
[1,3,5,7,9]
target = 7

Output = 3
```

★ Brute Force Intuition

Simply check every element.

```
1 → 3 → 5 → 7
```

Worst case:

```
O(N)
```

★ Optimal Intuition

Because the array is sorted, look at the middle.

Suppose:

```
[1 3 5 7 9 11 13]
      ↑
    mid
```

If mid is smaller than the target:

Everything left of mid is also smaller.

So discard the left half.

If mid is larger:

Everything right of mid is also larger.

Discard the right half.

Every step removes half the search space.

Optimal — $O(\log N)$

```
int search(vector<int>& a,int target)
{
    int l=0,r=a.size()-1;

    while(l<=r)
    {
        int mid=l+(r-l)/2;

        if(a[mid]==target)
            return mid;

        if(a[mid]<target)
            l=mid+1;
        else
            r=mid-1;
    }

    return -1;
}
```

Core Insight

Sorted + can eliminate half → Binary Search.

2. First and Last Position of Element

LeetCode: 34

Problem

Find the first and last position of a target in a sorted array.

```
[5,7,7,8,8,10]
target = 8
```

```
Output:
[3,4]
```

★ Brute Force Intuition

Traverse the array and record:

- first occurrence
- last occurrence

Time: $O(N)$.

★ Optimal Intuition

A normal binary search stops as soon as it finds the target.

But here, finding an occurrence isn't enough.

For the first occurrence:

```
target found
  ↓
store answer
  ↓
continue LEFT
```

For the last occurrence:

```
target found
  ↓
store answer
  ↓
continue RIGHT
```

So binary search becomes a boundary-finding tool.

Optimal — $O(\log N)$

```
int first(vector<int>& a,int target)
{
    int l=0,r=a.size()-1,ans=-1;

    while(l<=r)
    {
        int mid=l+(r-l)/2;

        if(a[mid]==target)
        {
            ans=mid;
            r=mid-1;
        }
        else if(a[mid]<target)
            l=mid+1;
        else
            r=mid-1;
    }

    return ans;
}

int last(vector<int>& a,int target)
{
    int l=0,r=a.size()-1,ans=-1;

    while(l<=r)
    {
        int mid=l+(r-l)/2;

        if(a[mid]==target)
        {
            ans=mid;
            l=mid+1;
        }
        else if(a[mid]<target)
            l=mid+1;
        else
            r=mid-1;
    }
}
```



```
}  
  
    return ans;  
}
```

Core Insight

Binary Search doesn't have to find an element; it can find a boundary.

3. Search in Rotated Sorted Array

LeetCode: 33

Problem

Search for a target in a sorted array that has been rotated.

```
[4,5,6,7,0,1,2]  
target = 0  
  
Output = 4
```

★ Brute Force Intuition

Simply traverse the array.

Time: $O(N)$.

★ Optimal Intuition

The array isn't completely sorted, but there is a very important property:

At least one half around mid is always sorted.

Example:

```
[4,5,6,7,0,1,2]  
  L   M       R
```

Left half:

```
[4,5,6,7]
```

is sorted.

Now ask:

Is the target inside this sorted half?

If yes, search there.

Otherwise search the other half.

If the right half is sorted, do the same reasoning.

Optimal — $O(\log N)$

```
int search(vector<int>& a,int target)
```

```

{
    int l=0,r=a.size()-1;

    while(l<=r)
    {
        int mid=l+(r-l)/2;

        if(a[mid]==target)
            return mid;

        // Left half sorted
        if(a[l]<=a[mid])
        {
            if(a[l]<=target && target<a[mid])
                r=mid-1;
            else
                l=mid+1;
        }

        // Right half sorted
        else
        {
            if(a[mid]<target && target<=a[r])
                l=mid+1;
            else
                r=mid-1;
        }
    }

    return -1;
}

```

Core Insight

Rotated sorted array → identify the sorted half → check whether target belongs there.

4. Find Minimum in Rotated Sorted Array

LeetCode: 153

Problem

Find the minimum element in a rotated sorted array.

[4,5,6,7,0,1,2]

Output = 0

★ Brute Force Intuition

Traverse the entire array and maintain the minimum.

```

int findMin(vector<int>& a)
{
    int ans=INT_MAX;

    for(int x:a)
        ans=min(ans,x);

    return ans;
}

```

Time: O(N).

★ Optimal Intuition

Again, use the sorted structure.

Compare:

```
a[mid] with a[r]
```

If:

```
a[mid] > a[r]
```

The minimum must be to the right of mid.

Why?

Because mid is in the larger sorted section before the rotation point.

So:

```
l = mid + 1
```

Otherwise:

```
a[mid] <= a[r]
```

The right half is sorted, so the minimum could be mid itself or somewhere to its left.

Therefore:

```
r = mid
```

Optimal — $O(\log N)$

```
int findMin(vector<int>& a)
{
    int l=0,r=a.size()-1;

    while(l<r)
    {
        int mid=l+(r-l)/2;

        if(a[mid]>a[r])
            l=mid+1;
        else
            r=mid;
    }

    return a[l];
}
```

🧠 Core Insight

Compare middle with the right boundary to determine which side contains the rotation point.

5. Koko Eating Bananas

LeetCode: 875

Problem

Koko has piles of bananas and must finish them within h hours.

Find the minimum eating speed.

Example:

```
piles = [3,6,7,11]
h = 8
```

```
Answer = 4
```

★ Why this is Binary Search

The array isn't sorted.

The answer is the eating speed.

Possible speeds:

```
1 2 3 4 5 6 ... 11
```

For each speed, ask:

Can Koko finish within h hours?

The answer looks like:

```
Speed:
1 2 3 4 5 6 7 ...
X X X ✓ ✓ ✓ ✓
```

This is a monotonic condition.

Once a speed works, every larger speed also works.

Therefore we can binary search the answer.

Brute Intuition

Try every possible speed from:

```
1 → max(piles)
```

and find the first valid speed.

Time: $O(N \times \text{maxPile})$.

★ Optimal Intuition

Binary search speed.

For a speed k , hours needed for one pile:

```
[pile/k]
```

In C++:

```
(pile+k-1)/k
```

If total hours $\leq h$:

speed works $\rightarrow$ try smaller

Otherwise:

speed too slow $\rightarrow$ increase speed

Optimal — $O(N \log(\max\text{Pile}))$

```
int minEatingSpeed(vector<int>& piles,int h)
{
    int l=1;
    int r=*max_element(piles.begin(),piles.end());

    while(l<r)
    {
        int mid=l+(r-l)/2;

        long long hours=0;

        for(int pile:piles)
            hours+=(pile+mid-1)/mid;

        if(hours<=h)
            r=mid;
        else
            l=mid+1;
    }

    return l;
}
```

Core Insight

If you can check whether an answer is valid in $O(N)$ and validity is monotonic, binary search the answer.

6. Capacity to Ship Packages Within D Days

LeetCode: 1011

Problem

Find the minimum ship capacity needed to ship all packages within days.

```
weights = [1,2,3,4,5,6,7,8,9,10]
days = 5

Answer = 15
```

★ Why Binary Search?

Again, the array itself isn't the thing we're searching.

We're searching the capacity.

Minimum possible capacity:

```
max(weights)
```

because the ship must carry the heaviest package.

Maximum possible capacity:

```
sum(weights)
```

because we could ship everything in one day.

So:

```
Capacity:
10 11 12 ... 15 16 17 ...
X  X  X  ✓  ✓  ✓
```

Once a capacity works, any larger capacity also works.

That's the monotonic property.

Brute Intuition

Try every capacity from:

```
max(weights) → sum(weights)
```

For each capacity, calculate how many days are required.

Take the first capacity that works.

★ Optimal Intuition

Binary search capacity.

For a chosen capacity:

```
Keep adding packages to current day.

If adding next package exceeds capacity:
    start a new day.
```

Then check:

```
daysNeeded <= D
```

If yes:

capacity works → try smaller

If no:

capacity too small → increase

Optimal — $O(N \log(\text{sum}))$

```
int shipWithinDays(vector<int>& weights, int days)
{
    int l = *max_element(weights.begin(), weights.end());
```

```

int r=accumulate(weights.begin(),
                 weights.end(),0);

while(l<r)
{
    int mid=l+(r-l)/2;

    int daysNeeded=1;
    int current=0;

    for(int w:weights)
    {
        if(current+w>mid)
        {
            daysNeeded++;
            current=0;
        }

        current+=w;
    }

    if(daysNeeded<=days)
        r=mid;
    else
        l=mid+1;
}

return l;
}

```

Core Insight

Minimum possible answer + maximum possible answer + monotonic feasibility → Binary Search on Answer.

Chapter 7 — Pattern Recognition

This is the most important section.

Pattern 1: Sorted Array

```

Sorted?
↓
Can eliminate half?
↓
Binary Search

```

Pattern 2: First/Last Occurrence

```

Found target
↓
Don't stop
↓
Continue toward boundary

```

Pattern 3: Rotated Sorted Array

```

At least one half is sorted
↓
Identify sorted half
↓
Check target
↓

```

Discard one half

Pattern 4: Minimum in Rotated Array

Compare mid with right

↓

Find which side contains pivot/minimum

Pattern 5: Binary Search on Answer ★

Look for:

Minimum/Maximum answer

+

Can I check a candidate answer?

+

Valid answers are monotonic

Then:

Binary Search the answer

1. Bubble Sort

Intuition

Think of bubbles rising to the surface.

Compare adjacent elements:

```
5  3  8  4
^  ^
```

If they are in the wrong order, swap them. After one complete pass, the largest element reaches the end.

Example

[5, 3, 8, 4, 2]

Pass 1:

5 3 -> swap -> 3 5

5 8 -> no swap

8 4 -> swap -> 4 8

8 2 -> swap -> 2 8

[3, 5, 4, 2, 8]

8 is now fixed.

Repeat.

[3, 5, 4, 2, 8]

[3, 4, 2, 5, 8]

[3, 2, 4, 5, 8]

[2, 3, 4, 5, 8]

Code

```
void bubbleSort(vector<int>& a) {
    int n = a.size();
    for(int i = 0; i < n-1; i++) {
        bool swapped = false;
        for(int j = 0; j < n-i-1; j++) {
            if(a[j] > a[j+1]) {
                swap(a[j], a[j+1]);
                swapped = true;
            }
        }
        if(!swapped)
            break;
    }
}
```

Complexity

Case	Time
Best	$O(n)$
Average	$O(n^2)$

Worst	$O(n^2)$
-------	----------

Space: $O(1)$ Stable: Yes

2. Selection Sort

Intuition

At every step, find the minimum element from the unsorted portion and put it at the beginning.

[5, 3, 8, 4, 2]

^

sorted portion

Find minimum = 2.

Swap:

[2, 3, 8, 4, 5]

Then find minimum from: [3, 8, 4, 5] and continue.

Example

[5, 3, 8, 4, 2]

Minimum = 2

[2, 3, 8, 4, 5]

Minimum in remaining = 3

[2, 3, 8, 4, 5]

Minimum = 4

[2, 3, 4, 8, 5]

Minimum = 5

[2, 3, 4, 5, 8]

Code

```
void selectionSort(vector<int>& a) {  
    int n = a.size();  
    for(int i = 0; i < n-1; i++) {  
        int minIndex = i;  
        for(int j = i+1; j < n; j++) {  
            if(a[j] < a[minIndex])  
                minIndex = j;  
        }  
        swap(a[i], a[minIndex]);  
    }  
}
```

Complexity

Best: $O(n^2)$ Average: $O(n^2)$ Worst: $O(n^2)$

Space: $O(1)$ Stable: No

3. Insertion Sort

Intuition

Think about arranging playing cards in your hand. You pick one card and insert it into its correct position among the cards already sorted.

[5 | 3 8 4 2]

5 is sorted.

Take 3:

[3 5 | 8 4 2]

Take 8:

[3 5 8 | 4 2]

Take 4:

[3 4 5 8 | 2]

Finally:

[2 3 4 5 8]

Code

```
void insertionSort(vector<int>& a) {  
    int n = a.size();  
    for(int i = 1; i < n; i++) {  
        int key = a[i];  
        int j = i-1;  
        while(j >= 0 && a[j] > key) {  
            a[j+1] = a[j];  
            j--;  
        }  
        a[j+1] = key;  
    }  
}
```

Complexity

Case	Time
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$

Space: $O(1)$ Stable: Yes

When useful?

Insertion sort is surprisingly useful when the array is: already almost sorted, small, or being sorted incrementally.

4. Merge Sort ■■■■

This is very important.

Pattern: Divide and Conquer

Intuition

Instead of trying to sort the whole array at once:

[5, 3, 8, 4, 2, 7, 1, 6]

divide it:

[5,3,8,4] [2,7,1,6]

Again:

[5,3] [8,4] [2,7] [1,6]

Again:

[5] [3] [8] [4] [2] [7] [1] [6]

A single element is already sorted.

Now merge sorted pieces.

[5] [3]
|
[3,5]

Then:

[8] [4]
|
[4,8]

Eventually:

[3,5,4,8]
|
[3,4,5,8]

Code

```
void merge(vector<int>& a, int low, int mid, int high) {  
    vector<int> temp;  
  
    int i = low;  
    int j = mid+1;  
  
    while(i <= mid && j <= high) {  
        if(a[i] <= a[j])  
            temp.push_back(a[i++]);  
        else  
            temp.push_back(a[j++]);  
    }  
  
    while(i <= mid)  
        temp.push_back(a[i++]);  
  
    while(j <= high)  
        temp.push_back(a[j++]);  
  
    for(int k = low; k <= high; k++)  
        a[k] = temp[k-low];  
}  
  
void mergeSort(vector<int>& a, int low, int high) {  
    if(low >= high)  
        return;  
  
    int mid = low + (high-low)/2;  
  
    mergeSort(a, low, mid);  
    mergeSort(a, mid+1, high);  
  
    merge(a, low, mid, high);  
}
```

Call: `mergeSort(a, 0, a.size()-1);`

Example

[5,3,8,4]

Divide:

[5,3] [8,4]

Divide:

[5] [3] [8] [4]

Merge:

[3,5] [4,8]

Merge:

[3,4,5,8]

Complexity

Best: $O(n \log n)$ Average: $O(n \log n)$ Worst: $O(n \log n)$

Space: $O(n)$ Stable: Yes

Important DSA applications

Merge sort is not just for sorting. It is used for: Count Inversions, Reverse Pairs, Count Smaller Elements, Divide-and-conquer problems.

5. Quick Sort ■■■

Pattern: Divide and Conquer + Partition

Intuition

Choose a pivot. Put elements smaller than pivot to the left, elements greater than pivot to the right. Then recursively sort both sides.

Example:

[5,3,8,4,2,7,1,6]

Choose pivot = 5.

After partition:

[3,4,2,1] 5 [8,7,6]

Now recursively sort:

[3,4,2,1]

and:

[8,7,6]

Code — Lomuto Partition

```
int partition(vector<int>& a, int low, int high) {
    int pivot = a[high];
    int i = low-1;
    for(int j = low; j < high; j++) {
        if(a[j] < pivot) {
            i++;
        }
    }
}
```

```

        swap(a[i], a[j]);
    }
}

swap(a[i+1], a[high]);

return i+1;
}

void quickSort(vector<int>& a, int low, int high) {

    if(low >= high)
        return;

    int p = partition(a, low, high);

    quickSort(a, low, p-1);
    quickSort(a, p+1, high);
}

Call: quickSort(a, 0, a.size()-1);

```

Example

[5, 3, 8, 4, 2]

Pivot = 2

[2, 3, 8, 4, 5]

^

pivot fixed

Then recursively sort remaining portion.

Complexity

Case	Time
Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$

Space: $O(\log n)$ average recursion stack. Stable: No

Why worst case?

If the pivot repeatedly becomes the smallest/largest:

[1, 2, 3, 4, 5]

you get:

```

1 | [2, 3, 4, 5]
  | [3, 4, 5]
  ...

```

which becomes $O(n^2)$.

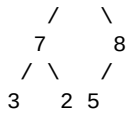
6. Heap Sort ■■■

Pattern: Heap

Intuition

For ascending sorting, construct a Max Heap. Max heap property: parent $\geq$ children.

Example:



The largest element is always at the root.

Take the root and put it at the end.

Then rebuild the heap.

Important operation: Heapify

```

void heapify(vector<int>& a, int n, int i) {
    int largest = i;

    int left = 2*i + 1;
    int right = 2*i + 2;

    if(left < n && a[left] > a[largest])
        largest = left;

    if(right < n && a[right] > a[largest])
        largest = right;

    if(largest != i) {
        swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}

```

Heap Sort

```

void heapSort(vector<int>& a) {
    int n = a.size();

    // Build Max Heap
    for(int i = n/2 - 1; i >= 0; i--)
        heapify(a, n, i);

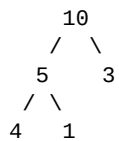
    // Extract maximum
    for(int i = n-1; i > 0; i--) {
        swap(a[0], a[i]);
        heapify(a, i, 0);
    }
}

```

Example

[4,10,3,5,1]

Build max heap:



Move 10 to end:

[1,5,3,4,10]

Heapify remaining:

[5,4,3,1,10]

Continue.

Final:

[1,3,4,5,10]

Complexity

Best: $O(n \log n)$ Average: $O(n \log n)$ Worst: $O(n \log n)$

Space: $O(1)$ auxiliary Stable: No

7. Counting Sort

When to use?

When values lie in a small range.

Example:

[4,2,2,8,3,3,1]

Maximum = 8.

Create frequency array:

value:	0	1	2	3	4	5	6	7	8
freq:	0	1	2	2	1	0	0	0	1

Then reconstruct.

Code

```
void countingSort(vector<int>& a) {  
    int maxVal = *max_element(a.begin(), a.end());  
    vector<int> freq(maxVal+1, 0);  
    for(int x : a)  
        freq[x]++;  
    int index = 0;  
    for(int x = 0; x <= maxVal; x++) {  
        while(freq[x]--) {  
            a[index++] = x;  
        }  
    }  
}
```

Example

Input:

[4,2,2,8,3,3,1]

Frequency:

1	->	1
2	->	2
3	->	2
4	->	1
8	->	1

Output:

[1,2,2,3,3,4,8]

Complexity

If range is k : Time: $O(n + k)$ Space: $O(k)$

Important limitation

If: [1, 2, 999999999] counting sort becomes inefficient because the range is huge.

8. Radix Sort

Intuition

Instead of comparing complete numbers, sort numbers digit by digit. Usually start with units, then tens, then hundreds, and so on.

Example:

[170, 45, 75, 90, 802, 24, 2, 66]

Sort by units:

[170, 90, 802, 2, 24, 45, 75, 66]

Then tens.

Then hundreds.

Final:

[2, 24, 45, 66, 75, 90, 170, 802]

Radix sort normally uses stable Counting Sort for each digit.

Code

```
void countingSort(vector<int>& a, int exp) {
    int n = a.size();

    vector<int> output(n);
    int count[10] = {};

    for(int x : a) {
        int digit = (x / exp) % 10;
        count[digit]++;
    }

    for(int i = 1; i < 10; i++)
        count[i] += count[i-1];

    for(int i = n-1; i >= 0; i--) {
        int digit = (a[i] / exp) % 10;

        output[count[digit]-1] = a[i];

        count[digit]--;
    }

    a = output;
}

void radixSort(vector<int>& a) {
    int maxVal = *max_element(a.begin(), a.end());

    for(int exp = 1;
        maxVal / exp > 0;
        exp *= 10) {
```

```

        countingSort(a, exp);
    }
}

```

Complexity

If n = number of elements, d = number of digits: Time: $O(nd)$ Space: $O(n + 10)$

9. Bucket Sort

Intuition

Divide elements into several buckets based on their value.

Example:

[0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68]

Create buckets:

Bucket 0 -> 0.0 - 0.1
 Bucket 1 -> 0.1 - 0.2
 Bucket 2 -> 0.2 - 0.3
 ...

Put each element in its bucket.

Sort individual buckets.

Then concatenate them.

Code

For numbers in [0,1):

```

void bucketSort(vector<float>& a) {
    int n = a.size();
    vector<vector<float>> buckets(n);

    for(float x : a) {
        int index = x * n;
        buckets[index].push_back(x);
    }

    for(auto& bucket : buckets)
        sort(bucket.begin(), bucket.end());

    int index = 0;
    for(auto& bucket : buckets) {
        for(float x : bucket)
            a[index++] = x;
    }
}

```

Example

Input:

[0.42, 0.32, 0.23, 0.52, 0.25]

Buckets:

0.2 -> 0.23, 0.25
 0.3 -> 0.32
 0.4 -> 0.42

0.5 -> 0.52

Output:

[0.23, 0.25, 0.32, 0.42, 0.52]

Complexity

Average: $O(n + k)$ under suitable distribution. Worst: $O(n^2)$ depending on bucket distribution and internal sorting.

■ Most Important Comparison

Algorithm	Best	Average	Worst	Space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^*$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Can be
Radix	$O(nd)$	$O(nd)$	$O(nd)$	$O(n+k)$	Yes
Bucket	$O(n+k)^{**}$	$O(n+k)^{**}$	$O(n^2)$	$O(n+k)$	Depends

* Quick Sort recursion stack; worst case can become $O(n)$. ** Under suitable distribution assumptions.

■■ What You Actually Need for DSA

Don't spend equal time on all of them.

Must Know Completely

1. Merge Sort

Know:

Divide

|

Sort left

|

Sort right

|

Merge

And applications: Count Inversions, Reverse Pairs

2. Quick Sort

Know:

Pivot

|

Partition

|

Left + Pivot + Right

|

Recursion

Especially understand why worst case becomes $O(n^2)$.

3. Heap Sort

Know:

Build Heap

|

Maximum at root

|
Swap root with last
|
Heapify
|
Repeat

This also connects directly to Priority Queue / Heap problems.

Know the basics

Bubble → Selection → Insertion. You mainly need their intuition and complexity.

Know conceptually

Counting → Radix → Bucket. These are less frequently required in normal interview coding, but they are important for understanding non-comparison sorting.

■ One Very Important Concept

Sorting algorithms can be divided into:

Comparison-based

They decide order by comparing elements: $a[i] < a[j]$

Examples: Bubble, Selection, Insertion, Merge, Quick, Heap

Their general lower bound is: $\Omega(n \log n)$ for comparison sorting.

Non-comparison-based

They exploit the values themselves: Counting, Radix, Bucket

They can achieve better than $O(n \log n)$ under suitable assumptions.

■■ Quick Recognition Trick

"Count inversions" → **Merge Sort**

"Reverse pairs" → **Modified Merge Sort**

"Need guaranteed $O(n \log n)$, $O(1)$ extra space" → **Heap Sort**

"Partition around pivot" → **Quick Sort**

"Small range of integer values" → **Counting Sort**

"Sort digit by digit" → **Radix Sort**

"Values uniformly distributed" → **Bucket Sort**

"Nearly sorted array" → **Insertion Sort**